Software Architecture & Design Principles

Doc Ref: ACQO-DOC-003 | v1.0 | 2025

Autonomous Code Quality & Optimization Intelligence System

1. Architecture Overview

The ACQO system monitors four repositories spanning microservices, data processing, and web application architectures. This document defines the architectural principles, design patterns, and scalability guidelines that govern all repositories and form the baseline for architecture-level code reviews.

2. Repository Architecture Types

Repository	Language	Architecture	Team Owner	Key Design Concern
payment-service	Java	Microservices	Platform Team	Transaction consistency & idempotency
analytics-engine	Python	Data Processing	Data Team	Pipeline throughput & memory efficiency
user-auth-service	Java	Microservices	Security Team	Zero-trust auth & token lifecycle
reporting-dashboard	JavaScript	Web Application	Frontend Team	Render performance & API contract stability

3. Layered Architecture Principles

All microservice repositories (payment-service, user-auth-service) must follow a strict layered architecture to maintain separation of concerns and testability:

Layer	Responsibility	Allowed Dependencies	Prohibited
API / Controller	Request handling, input validation, routing	Service layer only	Direct DB access or business logic
Service / Domain	Business logic, orchestration, domain rules	Repository layer, domain models	HTTP/framework imports
Repository / DAO	Data persistence, query execution	ORM / JDBC, domain models	Business logic, HTTP calls
Infrastructure	External integrations, messaging, caching	Service layer (injected)	Direct domain rule enforcement

4. SOLID Principles — Application Reference

Principle	Definition	Current Violation Example	Remediation
Single Responsibility	One class = one reason to change	PaymentProcessor.java handles auth + payment + logging	Split into focused classes
Open / Closed	Open for extension, closed for modification	Switch statements for payment type routing	Use Strategy pattern
Liskov Substitution	Subtypes must be substitutable for base types	AuthManager override breaks interface contract	Enforce interface contracts

Principle	Definition	Current Violation Example	Remediation
Interface Segregation	Clients should not depend on unused interfaces	Large IPaymentService with 18 methods	Split into focused interfaces
Dependency Inversion	Depend on abstractions, not concretions	Direct instantiation of DB client in service	Inject via constructor / DI container

5. Microservices Design Patterns

- **API Gateway:** A single entry point routes and authenticates all inbound requests before they reach payment-service or auth-service. Centralises cross-cutting concerns.
- **Circuit Breaker:** Wrap downstream calls in circuit breakers (Resilience4j) to prevent cascading failures when dependencies degrade.
- **Saga Pattern:** For distributed transactions in payment-service, use choreography-based sagas with compensating transactions rather than distributed 2PC.
- **Outbox Pattern:** Guarantee event delivery by writing events to an outbox table within the same DB transaction before publishing to Kafka.
- **Sidecar / Service Mesh:** Delegate observability (tracing, metrics, retries) to a sidecar proxy (Envoy/Istio) to keep service code focused on business logic.

6. Event-Driven Architecture Guidelines

Component	Pattern	Technology	Key Rule
Payment events	Async publish after commit	Kafka	At-least-once delivery; idempotent consumers
Analytics pipeline	Stream processing	Kafka + Flink	Backpressure handling mandatory
Auth events (audit)	Event sourcing (append-only)	Kafka	Never mutate; replay from log
Dashboard updates	Server-sent events / WebSocket	Node.js	Reconnect logic on client required

7. System Scalability Patterns

The following scalability patterns are approved for use across ACQO-monitored services:

Pattern	Use Case	Repositories	Scaling Dimension
Horizontal Pod Autoscaling	Stateless services under variable load	payment-service, auth-service	Request throughput
Read Replica Routing	Read-heavy dashboard queries	reporting-dashboard	DB read throughput
Sharding	High-volume analytics data partitioning	analytics-engine	Storage & compute
Async Job Queue	CPU-intensive background processing	analytics-engine	Latency isolation
CDN Offloading	Static asset and cached API delivery	reporting-dashboard	Network & origin load

Architecture Review Workflow

✔ Recommended Practices	✘ Prohibited Practices
✓ Enforce strict layer boundaries via dependency rule checks	✗ Allow cross-layer dependencies to shortcut development time
✓ Use circuit breakers on all inter-service HTTP calls	✗ Use distributed transactions (2PC) across microservices
✓ Apply SOLID principles during design review, not after	✗ Skip architecture review for "small" changes that touch interfaces